

output buffer. When the interpreter encounters a `ReadLine`, it can pop an element off the input buffer, and when it encounters a `PrintLine(s)`, it can push `s` onto the output buffer:

```
sealed trait Console[A] {
  ...
  def toState: ConsoleState[A]
}
case class Buffers(in: List[String], out: List[String])
case class ConsoleState[A](run: Buffers => (A, Buffers)) { ... }

object ConsoleState {
  implicit val monad: Monad[ConsoleState] = ...
}

val consoleToState = new (Console ~> ConsoleState) {
  def apply[A](a: Console[A]) = a.toState
}

def runConsoleState[A](io: ConsoleIO[A]): ConsoleState[A] =
  runFree[Console, ConsoleState, A](io)(consoleToState)
```

Represents a pair of buffers. The `in` buffer will be fed to `ReadLine` requests, and the `out` buffer will receive strings contained in `PrintLine` requests.

A specialized state action.

Converts to a pure state action.

This will allow us to have multiple interpreters for our little languages! We could, for example, use `runConsoleState` for testing console applications with our property-based testing library from chapter 8, and then use `runConsole` to run our program for real.¹³

The fact that we can write a generic `runFree` that turns `Free` programs into `State` or `Reader` values demonstrates something amazing—there’s nothing about our `Free` type that requires side effects of any kind. For example, from the perspective of our `ConsoleIO` programs, we can’t know (and don’t care) whether they’re going to be run with an interpreter that uses “real” side effects like `runConsole`, or one like `runConsoleState` that doesn’t. As far as we’re concerned, *a program is just a referentially transparent expression*—a pure computation that may occasionally make requests of some interpreter. The interpreter is free to use side effects or not. That’s now an entirely separate concern.

13.5 Non-blocking and asynchronous I/O

Let’s turn our attention now to the last remaining problem with our original `IO` monad—that of performing non-blocking or asynchronous I/O. When performing I/O, we frequently need to invoke operations that take a long time to complete and don’t occupy the CPU. These include accepting a network connection from a server socket, reading a chunk of bytes from an input stream, writing a large number of bytes to a file,

¹³ Note that `runConsoleReader` and `runConsoleState` aren’t stack-safe as implemented, for the same reason that `runConsoleFunction0` wasn’t stack-safe. We can fix this by changing the representations to `String => TailRec[A]` for `ConsoleReader` and `Buffers => TailRec[(A, Buffers)]` for `ConsoleState`.

and so on. Let's think about what this means in terms of the implementation of our `Free` interpreter.

When `runConsole`, for example, encounters a `Suspend(s)`, `s` will be of type `Console` and we'll have a translation `f` from `Console` to the target monad. To allow for non-blocking asynchronous I/O, we simply change the target monad from `Function0` to `Par` or another concurrency monad such as `scala.concurrent.Future`. So just like we were able to write both pure and effectful interpreters for `Console`, we can write both blocking and non-blocking interpreters as well, just by varying the target monad.¹⁴

Let's look at an example. Here, `runConsolePar` will turn the `Console` requests into `Par` actions and then combine them all into one `Par[A]`. We can think of it as a kind of compilation—we're replacing the abstract `Console` requests with more concrete `Par` requests that will actually read from and write to the standard input and output streams when the resulting `Par` value is run:

```
scala> def p: ConsoleIO[Unit] = for {
  |   _ <- println("What's your name?")
  |   n <- readLn
  |   _ <- n match {
  |     case Some(n) => println(s"Hello, $n!")
  |     case None => println(s"Fine, be that way.")
  |   }
  | } yield ()
p: ConsoleIO[Unit] =
FlatMap(Suspend(PrintLine(What's your name?)),<function1>)

scala> val q = runConsolePar(p)
q: Par[Unit] = <function1>
```

Although this simple example runs in `Par`, which in principle permits asynchronous actions, it doesn't make use of any asynchronous actions—`readLine` and `println` are both *blocking* I/O operations. But there are I/O libraries that support *non-blocking* I/O directly and `Par` will let us bind to such libraries. The details of these libraries vary, but to give the general idea, a non-blocking source of bytes might have an interface like this:

```
trait Source {
  def readBytes(
    numBytes: Int,
    callback: Either[Throwable, Array[Byte]] => Unit): Unit
}
```

Here it's assumed that `readBytes` returns immediately. We give `readBytes` a callback function indicating what to do when the result becomes available or the I/O subsystem encounters an error.

¹⁴ Our `Par` monad from chapter 7 doesn't do any handling of exceptions. See the file `Task.scala` in the answer source code accompanying this chapter for an example of an asynchronous I/O monad with proper exception handling.

Obviously, using this sort of library directly is painful.¹⁵ We want to program against a compositional monadic interface and abstract over the details of the underlying non-blocking I/O library. Luckily, the `Par` type lets us wrap these callbacks:

```
trait Future[+A] {
  private def apply(k: A => Unit): Unit
}

type Par[+A] = ExecutorService => Future[A]
```

The internal representation of `Future` is remarkably similar to that of `Source`. It's a single method that returns immediately, but takes a callback or continuation `k` that it will invoke once the value of type `A` becomes available. It's straightforward to wrap `Source.readBytes` in a `Future`, but we'll need to add a primitive to our `Par` algebra:¹⁶

```
def async[A](run: (A => Unit) => Unit): Par[A] = es => new Future {
  def apply(k: A => Unit) = run(k)
}
```

With this in place, we can now wrap the asynchronous `readBytes` function in the nice monadic interface of `Par`:

```
def nonblockingRead(source: Source, numBytes: Int):
  Par[Either[Throwable, Array[Byte]]] =
  async { (cb: Either[Throwable, Array[Byte]] => Unit) =>
    source.readBytes(numBytes, cb)
  }

def readPar(source: Source, numBytes: Int):
  Free[Par, Either[Throwable, Array[Byte]]] =
  Suspend(nonblockingRead(source, numBytes))
```

And we can now use regular for-comprehensions to construct chains of non-blocking computations:

```
val src: Source = ...
val prog: Free[Par, Unit] = for {
  chunk1 <- readPar(src, 1024)
  chunk2 <- readPar(src, 1024)
  ...
}
```



EXERCISE 13.5

Hard: We're not going to work through a full-fledged implementation of a non-blocking I/O library here, but you may be interested to explore this on your own by building off the

¹⁵ Even this API is rather nicer than what's offered directly by the `nio` package in Java (API at <http://mng.bz/uojM>), which supports non-blocking I/O.

¹⁶ This may in fact be the most primitive `Par` operation. The other primitives we developed for `Par` in chapter 7 could be implemented in terms of this one.